PaDiM: Model Optimization

From Research Code to Production-Ready Inference

549x Performance Improvement

March 2026

What is PaDiM & Why Does It Matter?

Why PaDiM is Important:

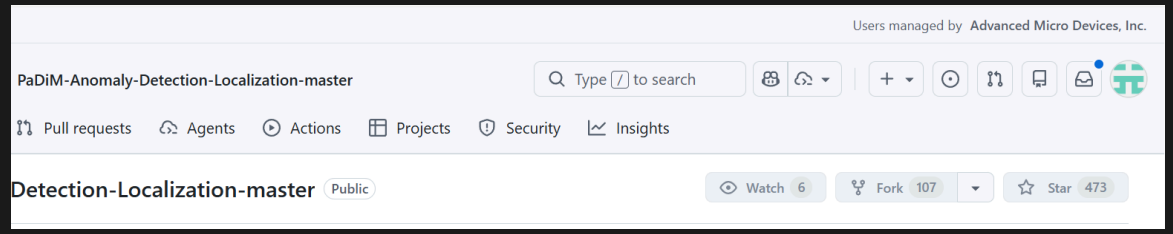
- **Zero-shot anomaly detection** - no -ve samples needed
- **Used in** Manufacturing QC, PCB inspection, Surface anomaly detection
- State-of-the-art - 96%+ AUROC on MVTec AD benchmark

How It Works:

1. **Learn "Normal"**: Extract features from +ve examples using CNN (ResNet) backbone
2. **Build Statistical Model**: Fit Gaussian distribution per patch location
3. **Detect Anomalies**: Measure deviation using Mahalanobis distance

The Challenge:

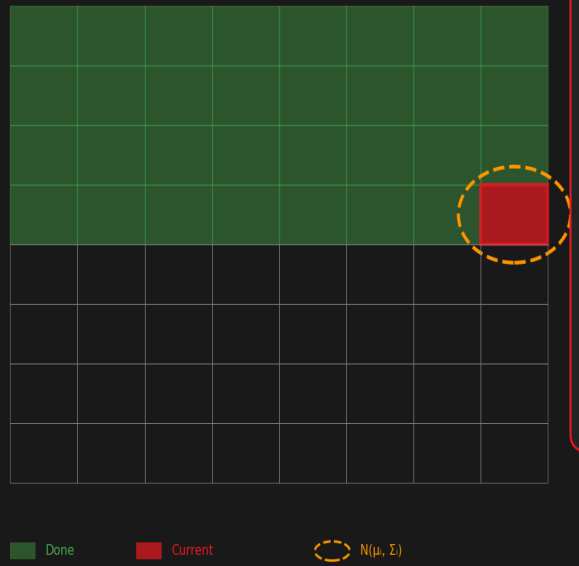
- Started from a popular open-source implementation.
- **Not production-ready** out of the box
- **Not designed for edge compute**
- A lot of Sequential processing, Algorithmic latency



PaDiM Pipeline: Gaussian Scanning Process

BEFORE: Sequential Loop

Position i = 32 of 3,136



LOOP ALGORITHM

For each position $i = 1, 2, ..., 3136$:

1. Extract feature vector x_i
2. Get learned mean μ_i
3. Get covariance matrix Σ_i
4. **Compute inverse $\Sigma_i^{-1} = \text{inv}(\Sigma_i)$** ← $O(n^3)$
5. Distance $d_i = \sqrt{[(x_i-\mu_i)^T \Sigma_i^{-1} (x_i-\mu_i)]}$

End loop

Inversions computed: **32** of 64

Total operations: **32,000,000**

621 ms per image | 1.6 FPS | TOO SLOW FOR REAL-TIME

Each patch compared against learned Gaussian distribution

Pipeline Steps:

1. Feature Extraction (ResNet18)
2. Embedding Concat (448D)
3. Dimension Reduction (→100D)
4. Gaussian Scan - Compare each patch to learned distribution
5. Anomaly Map - Mahalanobis distance per spatial location

The Bottleneck:

- 3,136 spatial positions
- Each requires 100×100 matrix inverse
- Matrix inverse: $O(n^3) = O(100^3)$
- Python loop with SciPy calls

The Bottleneck: Where Was Time Being Spent?

Baseline Time Breakdown

Component	Time	% of Total
Feature Extraction	15 ms	2.4%
Embedding Concat	12 ms	1.9%
Mahalanobis Loop	**590 ms**	**95%**
Post-processing	4 ms	0.6%
Total	**621 ms**	100%

Key Optimization Insight

Inverse Covariance Can Be Pre-computed

Original Approach (Per-Inference):

```
For each of 3,136 positions:
    1. Get covariance matrix Σ
    2. Compute inverse Σ⁻¹  ← SLOW!
    3. Calculate Mahalanobis distance
```

Solution:

- Σ^{-1} is **constant** after training
- Compute once, save to disk
- Load and reuse during inference

Impact: Eliminates 3,136 matrix inversions at runtime

Optimized Pipeline:

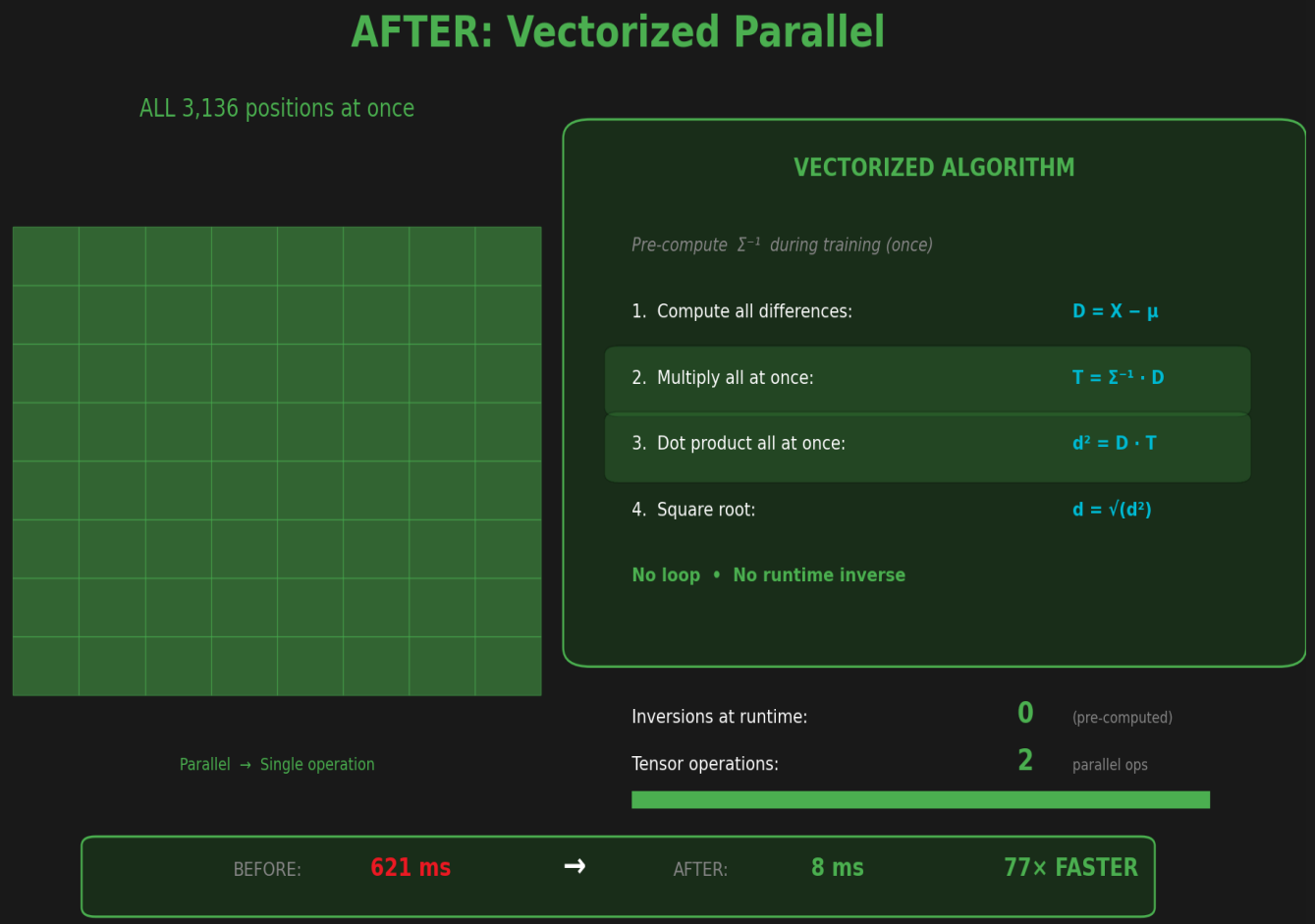
With `inv_cov` and `mean` pre-computed and loaded:

```
# Pre-loaded from training
mean = load("mean.pkl")         # [100, 56, 56]
inv_cov = load("inv_cov.pkl")  # [100, 100, 56, 56]

# Inference - no matrix inverse needed!
diff = embedding - mean
dist = mahalanobis(diff, inv_cov)
```

Result: The expensive operations are moved to training time. Rest of the loop can be **parallelized** and **converted to ONNX**.

Vectorized Mahalanobis Distance



Implementation (Einstein Summation):

```
tmp = np.einsum('cdi,bdi->bci', inv_cov, diff)
dist = np.sqrt(np.einsum('bci,bci->bi', diff, tmp))
```

- **Mahalanobis:** $d(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$
- **Vectorized:** $\text{dist}[i] = \sqrt{\sum_c \sum_d \text{diff}[c,i] \cdot \Sigma^{-1}[c,d,i] \cdot \text{diff}[d,i]}$
- **Key Insight:**
  - No Python loops
  - BLAS/LAPACK optimized, CPU SIMD (AVX2)
  - **ONNX convertible**
  - All 3,136 distances in ONE op

Performance Results

Speed Gains from Baseline to BF16 + Diagonal Covariance

Configuration	Time (ms)	FPS	vs Baseline
Baseline PyTorch CPU (scipy loop)	621.51	1.6	1.0x
+ Pre-computed Σ^{-1}	~80	~12	~8x
+ Vectorized einsum (CPU)	~36	~28	~17x
+ ONNX Runtime (CPU)	~15	~67	~41x
+ GPU MIGraphX (FP32 + Full Cov)	7.43	135	84x
+ GPU BF16 (Full Cov)	6.79	147	92x
+ GPU BF16 + Diagonal Cov	**1.14**	**878**	**549x**

Key Achievement: From 621 ms to 1.14 ms — **549x speedup** with no accuracy loss

Diagonal vs Full Covariance Comparison

The Final Optimization: 6x Additional Speedup

Full Covariance:

- Mahalanobis: $d(x) = \sqrt{(x-\mu)^T \Sigma^{-1} (x-\mu)}$
- Matrix: [100 × 100] per position
- Time: 6.27 ms (Mahalanobis only)

Diagonal Covariance:

- Mahalanobis: $d(x) = \sqrt{\sum_i (x_i - \mu_i)^2 / \sigma_i^2}$
- Vector: [100] per position
- Time: 0.62 ms (10x faster)

Mode	Total Time	FPS	AUROC
Diagonal	**1.14 ms**	**878**	98.14
Full	6.79 ms	147	98.14

Key Finding:

- **No accuracy loss**
- **6x faster** total pipeline

Summary

549x Faster | 1.14ms Inference | 878 FPS

Metric	Baseline	Optimized
Time	621 ms	**1.14 ms**
FPS	1.6	**878**
Speedup	-	**549x**
Accuracy	98.14%	**98.14%**

Future Optimization Opportunities

What Could Be Done Next

NPU Acceleration:

- Offload ResNet18 feature extraction
- Running CNN backbone on **NPU** to free up GPU

Batch Inference:

- Process multiple images in parallel
- N× throughput for batch size N

Current Status:

- **878 FPS** far exceeds real-time requirements
- Ready for edge deployment